

Python Interview Prep

Collections Collections · NeetCode Patterns · Language Constructs · Patterns Collections · NeetCode Patterns · Language Constructs · Language Collections · NeetCode Patterns · Language Constructs · Essentials

Staff / Senior reference | Python 3 | June 20, 2026

PART I — Data Structures & Collections

1. Master Complexity Table

Structure	Access	Search	Insert	Delete	Notes
<code>list</code>	$O(1)$	$O(n)$	append $O(1)^*$, insert(i) $O(n)$	pop() $O(1)$, pop(0) $O(n)$	contiguous array
<code>dict</code>	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	insertion-ordered (3.7+)
<code>set</code> / <code>frozenset</code>	—	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	hash-based, no order
<code>deque</code>	$O(n)$	$O(n)$	append/appendleft $O(1)$	pop/popleft $O(1)$	$O(1)$ both ends
<code>heapq</code> (list)	heap[0] $O(1)$	$O(n)$	heappush $O(\log n)$	heappop $O(\log n)$	heapify $O(n)$
<code>SortedList</code>	$O(\log n)$ by index	$O(\log n)$	$O(\log n)$	$O(\log n)$	third-party; ordered
<code>str</code>	$O(1)$	$O(n)$	immutable	immutable	build via list + join

* **amortized**. The two facts interviewers probe: `list.pop(0)` / `list.insert(0, x)` are $O(n)$ — use a deque; and `x in list` is $O(n)$ while `x in set` is $O(1)$.

2. list — methods & complexity

Method	Complexity	Note
<code>append(x)</code>	$O(1)$ amortized	stack push
<code>pop()</code>	$O(1)$	stack pop
<code>pop(i)</code>	$O(n)$	shifts tail
<code>insert(i, x)</code>	$O(n)$	shifts tail
<code>remove(x)</code>	$O(n)$	first match
<code>index(x)</code>	$O(n)$	raises if absent
<code>count(x)</code>	$O(n)$	
<code>x in arr</code>	$O(n)$	linear scan
<code>sort(key=, reverse=)</code>	$O(n \log n)$	in place, stable (Timsort)
<code>reverse()</code>	$O(n)$	in place
<code>extend(it)</code>	$O(k)$	
<code>arr[i:j]</code>	$O(j-i)$	copies

```
# 2D fill — NOT [[0]*cols]*rows, which aliases every row
grid = [[0] * cols for _ in range(rows)]

# multi-key sort, mixed direction
arr.sort(key=lambda p: (p[0], -p[1]))

# reversed copy
rev = arr[::-1]
```

3. dict — methods & complexity

Method	Complexity	Note
<code>d[k]</code> / <code>d[k]=v</code> / <code>del d[k]</code>	$O(1)$ avg	
<code>d.get(k, default)</code>	$O(1)$ avg	no <code>KeyError</code>
<code>d.setdefault(k, default)</code>	$O(1)$ avg	group without <code>defaultdict</code>
<code>d.pop(k, default)</code>	$O(1)$ avg	
<code>d.popitem()</code>	$O(1)$	LIFO (3.7+)
<code>k in d</code>	$O(1)$ avg	
<code>d.update(other)</code>	$O(\text{len other})$	
<code>d.keys/values/items()</code>	$O(1)$ view	lazy views
<code>d1 d2</code> (3.9+)	$O(\text{len both})$	merge; <code> =</code> to mutate

4. set / frozenset — methods & complexity

Method	Complexity	Note
<code>add(x)</code>	$O(1)$ avg	
<code>remove(x)</code>	$O(1)$ avg	<code>KeyError</code> if absent
<code>discard(x)</code>	$O(1)$ avg	silent if absent
<code>pop()</code>	$O(1)$	arbitrary element
<code>x in s</code>	$O(1)$ avg	
<code>a b</code> (union)	$O(\text{len a} + \text{len b})$	
<code>a & b</code> (intersection)	$O(\min(\text{len a}, \text{len b}))$	
<code>a - b</code> (difference)	$O(\text{len a})$	
<code>a ^ b</code> (sym. diff)	$O(\text{len a} + \text{len b})$	
<code>a <= b</code> (issubset)	$O(\text{len a})$	

`frozenset` is hashable — use it as a dict key or set element (e.g. canonical visited states).

5. str — key facts

Immutable. Building with `s += c` in a loop is $O(n^2)$; collect into a list and `"".join(parts)` for $O(n)$.
`ord('a')` / `chr(97)` for `char` $\leftrightarrow$ `int` bucket counting. `s.split()`, `s.rsplit(maxsplit=1)`.

6. collections.deque — methods & complexity

Method	Complexity	Note
<code>append</code> / <code>appendleft</code>	$O(1)$	
<code>pop</code> / <code>popleft</code>	$O(1)$	BFS uses <code>popleft</code>
<code>extend</code> / <code>extendleft</code>	$O(k)$	
<code>rotate(n)</code>	$O(\min(n, \text{len}))$	
<code>dq[i]</code>	$O(n)$	$O(1)$ only at ends
<code>remove(x)</code> / <code>index(x)</code>	$O(n)$	
<code>deque(maxlen=k)</code>	—	auto-evicts oldest

Use for: BFS, sliding-window, monotonic queue.

7. collections.defaultdict

Same complexity as `dict`; supplies a default via a factory on missing-key access. `defaultdict(list)` for adjacency lists, `defaultdict(int)` for counting, `defaultdict(set)` for grouping. Gotcha: read access *creates* the key — use plain `dict.get` if you must not mutate.

8. collections.Counter — methods & complexity

Method	Complexity	Note
<code>Counter(iterable)</code>	$O(n)$	frequency map
<code>most_common(k)</code>	$O(n \log k)$	$O(n \log n)$ if <code>k=None</code>
<code>elements()</code>	$O(\text{total count})$	
<code>update</code> / <code>subtract</code>	$O(n)$	
<code>c1 + c2</code> , <code>c1 - c2</code> , <code>c1 & c2</code> , <code>c1 \ c2</code>	$O(n)$	multiset arithmetic
<code>total()</code> (3.10+)	$O(n)$	sum of counts

`Counter(s1) == Counter(s2)` is a clean anagram check.

9. collections.OrderedDict — LRU backbone

Method	Complexity	Note
<code>move_to_end(k, last=True)</code>	$O(1)$	mark most-recently-used
<code>popitem(last=False)</code>	$O(1)$	evict least-recently-used
all dict ops	same as dict	

10. collections.namedtuple

Immutable lightweight record; field access by name. Readable heap entries: `Item = namedtuple("Item", "priority node")`. Zero method-complexity overhead vs a plain tuple.

11. heapq — functions & complexity

Function	Complexity	Note
<code>heapify(arr)</code>	$O(n)$	in place, min-heap
<code>heappush(h, x)</code>	$O(\log n)$	
<code>heappop(h)</code>	$O(\log n)$	
<code>h[0]</code>	$O(1)$	peek min
<code>heappushpop(h, x)</code>	$O(\log n)$	push then pop, one sift
<code>heapreplace(h, x)</code>	$O(\log n)$	pop then push
<code>nlargest/smallest(k, it)</code>	$O(n \log k)$	
<code>merge(*iters)</code>	$O(n \log k)$	merge k sorted

```
# max-heap: negate values on the way in and out
heapq.heappush(h, -x)
```

```
# tie-break with a counter so heap never compares the nodes themselves
heapq.heappush(h, (dist, next(counter), node))
```

12. bisect — functions & complexity

Function	Complexity	Note
<code>bisect_left(a, x)</code>	$O(\log n)$	first index $\geq x$
<code>bisect_right(a, x)</code>	$O(\log n)$	first index $> x$
<code>insort_left/right(a, x)</code>	$O(n)$	$\log n$ search + $O(n)$ shift

Count of x in sorted `a`: `bisect_right(a, x) - bisect_left(a, x)`. The $O(n)$ insert is exactly why repeated insertion into a sorted list is $O(n^2)$ — switch to SortedList below.

13. THIRD-PARTY: sortedcontainers (deep dive)

```
pip install sortedcontainers — from sortedcontainers import SortedList, SortedDict, SortedSet
```

How it actually works (the part that confuses people)

It is **NOT a balanced binary search tree**. Internally it is a **list of sorted sublists** (plus a "maxes" index of each sublist's largest element, and a positional index). To add a value: bisect the maxes to pick the target sublist ($O(\log(n/\text{load}))$), then `list.insert` into that sublist. Sublists split/merge to stay near a load factor (default 1000).

Consequence: the per-op sublist insert is technically $O(\sqrt{n})$ with a tiny constant, but bisect plus contiguous-memory C-level list ops make it **faster than tree structures in CPython**. For interviews, analyze add/remove/index as **$O(\log n)$** , but say out loud that it's a list-of-lists, not a tree — that candor signals you actually know the library.

SortedList — methods & complexity

Method	Complexity	Note
<code>add(x)</code>	$O(\log n)$	keeps sorted
<code>remove(x)</code> / <code>discard(x)</code>	$O(\log n)$	arbitrary element removal
<code>pop(i=-1)</code>	$O(\log n)$	
<code>sl[i]</code>	$O(\log n)$	indexed access — order statistics
<code>bisect_left/right(x)</code>	$O(\log n)$	
<code>index(x)</code> / <code>count(x)</code>	$O(\log n)$	
<code>x in sl</code>	$O(\log n)$	
<code>irange(lo, hi)</code>	$O(\log n + k)$	range iteration
<code>update(it)</code>	$O(k \log n)$	

SortedDict — methods & complexity (sorted by KEY)

Wraps a regular dict (values) + a SortedList of keys. Value lookup is dict-fast, but key insertion/deletion pays the sorted cost.

Method	Complexity	Note
<code>sd[k]</code> (read)	$O(1)$	dict hit
<code>sd[k] = v</code> (new key)	$O(\log n)$	insert into key list
<code>sd[k] = v</code> (existing)	$O(1)$	value overwrite
<code>del sd[k]</code>	$O(\log n)$	
<code>k in sd</code>	$O(1)$	
<code>peekitem(i)</code>	$O(\log n)$	i-th smallest key
<code>bisect_left/right(k)</code>	$O(\log n)$	on KEYS, not values
<code>irange(lo, hi)</code>	$O(\log n + k)$	

SortedSet — methods & complexity

Wraps a regular set (membership) + a SortedList (order). Membership is $O(1)$, but mutation pays the sorted cost.

Method	Complexity	Note
<code>x in ss</code>	$O(1)$	backed by set
<code>add(x)</code>	$O(\log n)$	SortedList insert
<code>discard(x)</code> / <code>remove(x)</code>	$O(\log n)$	
<code>ss[i]</code>	$O(\log n)$	indexed
<code>bisect_left/right(x)</code>	$O(\log n)$	
union / intersection / difference	$O(n \log n)$	

Which one: the decision that trips people up

Tool	Gives you	Cannot do	Use when
<code>heapq</code>	$O(1)$ peek min, $O(\log n)$ push/pop	no arbitrary remove, no ordered iteration, no indexing	top-K, streaming min/max, Dijkstra, merge-K
<code>bisect</code> + list	$O(\log n)$ search	$O(n)$ insert/delete (array shift)	set is static/small, or search-only
<code>SortedList</code>	$O(\log n)$ add/remove of ANY element + index + ordered iteration	slightly slower constants than <code>heapq</code> for pure top-K	dynamic ordered set: sliding-window median, kth-largest with removals, count-in-range over a changing set

Common mistakes

- Assuming `heapq` can remove or find an arbitrary element — it can't; you'd need lazy deletion (`push(value, version)`, skip stale entries on pop).
- Calling `bisect.insort` in a loop $\rightarrow O(n^2)$. Use `SortedList` for $O(n \log n)$.
- Mutating an element already inside a `SortedList` breaks ordering — remove, change, re-add.

- SortedDict `bisect` works on **keys**, not values.
- **Availability:** present on LeetCode; **NOT guaranteed** on CodeSignal / HackerRank / Codility. For a CodeSignal OA, prepare a heapq + lazy-deletion fallback, or hand-roll, before depending on SortedList.

14. itertools / functools quick hits

```
from itertools import accumulate, chain, combinations, permutations, product, pairwise

accumulate(nums)           # running prefix sums           O(n)
accumulate(nums, max)      # running max
pairwise(seq)               # adjacent pairs (3.10+)
product(range(3), repeat=2) # flattened nested loops / bitmask generation
combinations(arr, 2)
permutations(arr)
chain(a, b)                 # lazily concatenate iterables

from functools import cache, lru_cache, reduce, cmp_to_key

@cache                      # top-down DP memoization (3.9+); lru_cache(None) pre-3.9
def dp(i, j):
    ...

sorted(items, key=cmp_to_key(my_cmp)) # custom 3-way comparator
```


PART II — NeetCode / LeetCode Patterns

Each: **Cue** (when to reach for it) · template · complexity · representative problems.

1. Arrays & Hashing

Cue: counts, dedup, "have I seen this?". **$O(n)$ time / $O(n)$ space.**

```
seen = {}

for i, x in enumerate(nums):
    if target - x in seen:
        return [seen[target - x], i]
    seen[x] = i
```

Two Sum, Group Anagrams, Top-K Frequent, Contains Duplicate, Valid Anagram, Product of Array Except Self, Valid Sudoku, Longest Consecutive Sequence.

2. Two Pointers

Cue: sorted input; pair/triplet; palindrome; in-place dedup. **$O(n)$ after sort.**

```
l, r = 0, len(a) - 1

while l < r:
    s = a[l] + a[r]
    if s == target:
        return (l, r)
    if s < target:
        l += 1
    else:
        r -= 1
```

Valid Palindrome, Two Sum II, 3Sum, Container With Most Water, Trapping Rain Water.

3. Sliding Window

Cue: longest/shortest contiguous subarray or substring meeting a constraint. **$O(n)$.**

```
l = 0
window = {}

for r, c in enumerate(s):
    window[c] = window.get(c, 0) + 1

    # shrink from the left until the window is valid again
    while violates(window):
        window[s[l]] -= 1
        l += 1

    best = max(best, r - l + 1)
```

Fixed-size variant: slide a width-k frame, add `nums[r]`, drop `nums[r-k]`. *Best Time to Buy/Sell Stock, Longest Substring Without Repeating, Longest Repeating Char Replacement, Permutation in String, Minimum Window Substring, Sliding Window Maximum (monotonic deque).*

4. Stack / Monotonic Stack

Cue: matching/balancing; "next greater/smaller"; histogram spans. **$O(n)$** .

```
stack = [] # holds indices, values increasing

for i, x in enumerate(nums):
    while stack and nums[stack[-1]] < x:
        j = stack.pop()
        ans[j] = i - j # distance to next greater
    stack.append(i)
```

Valid Parentheses, Min Stack, Evaluate RPN, Generate Parentheses, Daily Temperatures, Car Fleet, Largest Rectangle in Histogram.

5. Binary Search (+ search-on-answer)

Cue: sorted array; OR "minimize the max / maximize the min / find a threshold". **$O(\log n)$** or **$O(n \log \text{range})$** .

```
lo, hi = min_ans, max_ans # search on the answer space

while lo < hi:
    mid = (lo + hi) // 2
    if feasible(mid):
        hi = mid
    else:
        lo = mid + 1

return lo
```

Binary Search, Search a 2D Matrix, Koko Eating Bananas, Find Minimum in Rotated, Search in Rotated, Time-Based Key-Value Store, Median of Two Sorted Arrays.

6. Linked List

Cue: cycle, middle, reorder, merge, k-group reversal. **$O(n)$** / **$O(1)$** space.

```
# in-place reversal
prev = None
while head:
    head.next, prev, head = prev, head, head.next

# fast/slow for cycle or middle
slow = fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
```

Reverse Linked List, Merge Two Lists, Linked List Cycle, Reorder List, Remove Nth From End, Copy List w/ Random Pointer, Add Two Numbers, LRU Cache, Merge K Sorted Lists, Reverse Nodes in k-Group.

7. Trees

Cue: traversal, path/depth, BST invariant. **O(n)**.

```
# depth / path aggregation
def dfs(node):
    if not node:
        return 0
    return 1 + max(dfs(node.left), dfs(node.right))

# BFS level-order
q = deque([root])
while q:
    for _ in range(len(q)):
        n = q.popleft()
        for c in (n.left, n.right):
            if c:
                q.append(c)
```

Invert Tree, Max Depth, Diameter, Balanced Tree, Same/Subtree, LCA of BST, Level Order, Right Side View, Count Good Nodes, Validate BST, Kth Smallest in BST, Construct from Preorder/Inorder, Serialize/Deserialize.

8. Tries

Cue: prefix search, word dictionary, autocomplete. **O(L)** per op.

```
root = {}

for w in words:
    node = root
    for ch in w:
        node = node.setdefault(ch, {})
    node['$'] = True  # end-of-word marker
```

Implement Trie, Design Add and Search Words, Word Search II.

9. Backtracking

Cue: "all subsets/permutations/combinations"; constraint satisfaction. **Exponential, pruned.**

```
def bt(start, path):
    res.append(path[:])  # record (or check goal)
    for i in range(start, n):
        if invalid(i):
            continue  # prune
        path.append(nums[i])
        bt(i + 1, path)
        path.pop()  # undo
```

Subsets (I/II), Combination Sum (I/II), Permutations, Word Search, Palindrome Partitioning, Letter Combinations, N-Queens.

10. Heap / Priority Queue / Top-K

Cue: k largest/smallest; merge-k; streaming median; schedule by frequency. $O(n \log k)$.

```
# two-heap running median (small = max-heap, large = min-heap)
small, large = [], []

def add(x):
    heapq.heappush(small, -heapq.heappushpop(large, x))
    if len(small) > len(large):
        heapq.heappush(large, -heapq.heappop(small))
```

Kth Largest in a Stream, Last Stone Weight, K Closest Points, Kth Largest Element (or quickselect), Task Scheduler, Design Twitter, Find Median from Data Stream.

11. Graphs (basic)

Cue: grid islands, connectivity, components, cycle detection. $O(V + E)$.

```
# Union-Find with path compression
def find(x):
    while x != par[x]:
        par[x] = par[par[x]]
        x = par[x]
    return x

def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb:
        return False
    par[ra] = rb
    return True
```

Number of Islands, Max Area of Island, Clone Graph, Pacific Atlantic, Surrounded Regions, Rotting Oranges, Walls and Gates, Course Schedule (I/II), Redundant Connection, Number of Connected Components, Graph Valid Tree, Word Ladder.

12. Advanced Graphs

Cue: weighted shortest path, dependency ordering, min spanning tree. $O(E \log V)$.

```
# Dijkstra
pq = [(0, src)]
dist = {}

while pq:
    d, u = heapq.heappop(pq)
    if u in dist:
        continue
    dist[u] = d
    for v, w in adj[u]:
        if v not in dist:
            heapq.heappush(pq, (d + w, v))
```

Kahn's topo sort: BFS over nodes with indegree 0 using a deque. *Network Delay Time*, *Reconstruct Itinerary*, *Min Cost to Connect Points (MST)*, *Swim in Rising Water*, *Alien Dictionary (topo)*, *Cheapest Flights within K Stops (Bellman-Ford)*.

13. 1-D Dynamic Programming

Cue: "ways to"; max/min along a sequence; subsequence. **$O(n)$ or $O(n \cdot \text{target})$.**

```
# House Robber shape
dp = [0] * (n + 1)

for i in range(1, n + 1):
    dp[i] = max(dp[i - 1], dp[i - 2] + val[i])
```

Climbing Stairs, *House Robber (I/II)*, *Longest Palindromic Substring*, *Palindromic Substrings*, *Decode Ways*, *Coin Change*, *Maximum Product Subarray*, *Word Break*, *Longest Increasing Subsequence*, *Partition Equal Subset Sum*.

14. 2-D Dynamic Programming

Cue: two strings; grid paths; knapsack; interval DP. **$O(m \cdot n)$.**

```
# Longest Common Subsequence
dp = [[0] * (n + 1) for _ in range(m + 1)]

for i in range(1, m + 1):
    for j in range(1, n + 1):
        if a[i - 1] == b[j - 1]:
            dp[i][j] = dp[i - 1][j - 1] + 1
        else:
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
```

Interval DP (solve inner ranges first): the family of **Burst Balloons** and the card-picking game — iterate by length, then by left endpoint, choosing the partition point. *Unique Paths*, *Longest Common Subsequence*, *Buy/Sell with Cooldown*, *Coin Change II*, *Target Sum*, *Interleaving String*, *Longest Increasing Path in Matrix*, *Distinct Subsequences*, *Edit Distance*, *Burst Balloons*, *Regular Expression Matching*.

15. Greedy

Cue: intervals; jumps; "minimum number of"; local choice provably global. **$O(n \log n)$ (sort).**

Maximum Subarray (Kadane), Jump Game (I/II), Gas Station, Hand of Straights, Merge Triplets to Form Target, Partition Labels, Valid Parenthesis String.

16. Intervals

Cue: meetings, merge, overlap counting. $O(n \log n)$.

```
intervals.sort()
merged = []

for s, e in intervals:
    if merged and s <= merged[-1][1]:
        merged[-1][1] = max(merged[-1][1], e)
    else:
        merged.append([s, e])
```

Min meeting rooms: min-heap of end times. *Insert Interval, Merge Intervals, Non-overlapping Intervals, Meeting Rooms (I/II), Minimum Interval to Include Each Query.*

17. Bit Manipulation

Cue: single number; count bits; subset enumeration via bitmask. $O(n)$ or $O(n \cdot \text{bits})$.

```
x ^= num          # XOR cancels pairs
count += n & 1     # inspect lowest bit
n >>= 1           # shift right
n &= (n - 1)       # drop the lowest set bit
```

Single Number, Number of 1 Bits, Counting Bits, Reverse Bits, Missing Number, Sum of Two Integers, Reverse Integer.

18. Math & Geometry

Cue: in-place matrix ops; overflow handling; gcd; prefix products.

Rotate Image, Spiral Matrix, Set Matrix Zeroes, Happy Number, Plus One, Pow(x, n), Multiply Strings, Detect Squares.

PART III — Python Language Constructs

1. Control Flow

```
if x > 0:
    ...
elif x < 0:
    ...
else:
    ...

val = a if cond else b           # ternary

for i, x in enumerate(arr, start=0):  # index + value
    ...
for a, b in zip(xs, ys):           # parallel iterate
    ...
for k, v in d.items():
    ...
for i in range(n - 1, -1, -1):     # reverse range
    ...

while l < r:
    ...

for x in seq:                     # loop-else: runs only if NO break
    if found(x):
        break
else:
    handle_not_found()

match cmd:                        # structural pattern matching (3.10+)
    case "start":
        ...
    case [a, b]:                  # sequence destructure
        ...
    case {"type": t}:             # mapping match
        ...
    case _:                       # default
        ...
```

2. Comprehensions

```
squares = [x * x for x in nums]
evens    = [x for x in nums if x % 2 == 0]
grid     = [[0] * cols for _ in range(rows)]  # 2D (not [[0]*cols]*rows)
pairs    = {k: v for k, v in items}           # dict comp
uniq     = {x for x in nums}                  # set comp
gen      = (x * x for x in nums)              # lazy generator expr
flat     = [c for row in matrix for c in row] # nested (outer loop first)
```

3. Functions

```
def f(a, b=0, *args, **kwargs):           # positional, default, varargs, kwargs
    ...

def g(acc=None):                          # AVOID mutable default (acc=[] persists)
    acc = acc if acc is not None else []

square = lambda x: x * x                  # anonymous
q, r = divmod(17, 5)                      # multiple returns via tuple unpack

def make_counter():                       # closure + nonlocal
    count = 0
    def inc():
        nonlocal count
        count += 1
        return count
    return inc
```

4. Error Handling

```
try:
    risky()
except (ValueError, KeyError) as e:       # catch specific, grouped
    handle(e)
except Exception as e:
    log(e)
    raise                                 # re-raise same exception
else:
    on_success()                          # runs only if no exception
finally:
    cleanup()                             # always runs

raise ValueError("bad input")
raise RuntimeError("wrap") from e        # exception chaining

class TaskNotFound(Exception):            # custom exception
    pass

assert n >= 0, "n must be non-negative"
```


Exception	Raised when
<code>IndexError</code>	list/sequence index out of range
<code>KeyError</code>	missing dict key
<code>ValueError</code>	right type, bad value (e.g. <code>int("x")</code>)
<code>TypeError</code>	wrong type / bad operands
<code>StopIteration</code>	iterator exhausted (next past end)
<code>ZeroDivisionError</code>	<code>/</code> or <code>%</code> by zero
<code>AttributeError</code>	missing attribute/method
<code>RecursionError</code>	recursion depth exceeded (~1000)

EAFP > LBYL in Python: prefer try/except over pre-checking (e.g. `try: d[k] except KeyError` rather than `if k in d`), except in tight loops where the exception path is hot.

5. Classes / OOP

```
class Node:
    __slots__ = ("val", "nxt")          # no __dict__: less memory, faster attr
    def __init__(self, val, nxt=None):
        self.val = val
        self.nxt = nxt
    def __repr__(self):                 # debug-friendly
        return f"Node({self.val})"
    def __lt__(self, other):            # enables heap / sort of objects
        return self.val < other.val
    def __eq__(self, other):
        return self.val == other.val
    def __hash__(self):                # define with __eq__ to use as set/dict key
        return hash(self.val)
```

Dunder methods worth knowing: `__init__`, `__repr__` / `__str__`, `__eq__` + `__hash__` (hashable key), `__lt__` (heap/sort), `__len__`, `__getitem__` / `__setitem__`, `__contains__` (the `in` operator), `__iter__` / `__next__`, `__call__`.

```

class Account:
    bank = "Acme"                                # class attr (shared across instances)
    def __init__(self, bal):
        self.bal = bal                            # instance attr
    @property
    def doubled(self):
        return self.bal * 2                       # computed, accessed as attr (no parens)
    @classmethod
    def from_cents(cls, c):
        return cls(c / 100)                       # alternate constructor
    @staticmethod
    def fee(x):
        return x * 0.01                           # no self / cls

class Savings(Account):                           # inheritance
    def __init__(self, bal, rate):
        super().__init__(bal)
        self.rate = rate

```

`dataclass` auto-generates `init/repr/eq`, and `order/lt` with `order=True` :

```

from dataclasses import dataclass, field

@dataclass(order=True)
class Point:
    x: int
    y: int = 0
    tags: list = field(default_factory=list)    # mutable default, done safely

```

Abstract base classes & Enum (your Strategy-pattern / sealed-interface designs map here):

```

from abc import ABC, abstractmethod

class PricingStrategy(ABC):
    @abstractmethod
    def price(self, ctx) -> float:
        ...

from enum import Enum

class Status(Enum):
    OPEN = 1
    CLOSED = 2

```

6. Iterators & Generators

```
def squares(n):
    for i in range(n):
        yield i * i                # lazy: produces one at a time

yield from other_gen()           # delegate to a sub-generator

class CountUp:                   # manual iterator protocol
    def __init__(self, n):
        self.i, self.n = 0, n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i >= self.n:
            raise StopIteration
        self.i += 1
        return self.i - 1
```

7. Decorators

```
from functools import wraps, cache

def logged(fn):
    @wraps(fn)                    # preserves name / docstring
    def wrapper(*args, **kwargs):
        return fn(*args, **kwargs)
    return wrapper

@cache                           # memoize -> top-down DP
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

8. Context Managers

```
with open("file.txt") as f:      # auto-closes even on exception
    data = f.read()

from contextlib import contextmanager

@contextmanager
def managed(res):
    acquire(res)
    try:
        yield res
    finally:
        release(res)             # guaranteed cleanup
```

9. Essential Built-ins & Idioms

```
enumerate(seq, start)      # index + value
zip(a, b)                  # parallel iterate
reversed(seq)
sorted(seq, key=fn, reverse=True)
min(seq, key=fn, default=0)
max(seq, key=fn, default=0)
any(conds)                 # short-circuits
all(conds)
sum(seq, start)
map(fn, seq)
filter(fn, seq)

a, b = b, a                # swap, no temp
first, *rest = seq         # extended unpacking
*init, last = seq

INF, NINF = float('inf'), float('-inf')
val = next(it, sentinel)   # safe next, no StopIteration

if (m := compute()) > 0:   # walrus := assigns inside an expression (3.8+)
    use(m)

import sys
sys.setrecursionlimit(10**6) # deep DFS
```

10. Type Hints (3.9+ style)

```
def topk(xs: list[int], k: int) -> list[int]:
    ...

def find(d: dict[str, int], k: str) -> int | None: # union / optional
    ...

from typing import Optional, Callable # Optional[int] == int | None
handler: Callable[[int], bool]
```

11. Python Gotchas (interview-critical)

- **Mutable default args** persist across calls — use a `None` sentinel, then assign inside.
- **Late-binding closures:** `[lambda: i for i in range(3)]` all return 2. Bind early: `lambda i=i: i`.
- **is vs ==:** `is` is identity; use it only for `is None`. Compare values with `==`.
- **Shallow copy:** nested lists share references; use `copy.deepcopy` for independent nesting.
- **Floor division:** `//` rounds toward $-\infty$ (`-7 // 2 == -4`); use `int(a / b)` to truncate.
- **defaultdict** read access creates the key — use plain `dict.get` when you must not mutate.
- **Recursion limit ~1000** — raise it or convert deep recursion to iterative with an explicit stack.
- **String concat in a loop** is $O(n^2)$ — collect into a list and `"".join(...)`.

PART IV — Python Essentials, OOP Deep Cuts & File I/O

1. Type Conversions & Division

```
int("42")           # str -> int
int("fff", 16)       # 255  parse with a base (also base 2, 8)
int(3.9)             # 3    float -> int TRUNCATES toward zero, not floor
int(-3.9)            # -3   (so int() differs from // for negatives)
int("3.5")           # ValueError! can't parse a float-looking string
int(float("3.5"))    # 3    go through float first

float("3.14")        # 3.14
float("1e3")         # 1000.0
float("inf")         # sentinel for DP / Dijkstra init; also float("-inf")

str(42)              # "42"
str([1, 2])          # "[1, 2]" <- the repr, almost never what you want
bool(0), bool(""), bool([]), bool(None)  # all False (the falsy set)

ord('a')             # 97   char -> int
chr(97)              # 'a'  int -> char
ord(c) - ord('a')    # 0..25 bucket index for lowercase counting
```

`bool` is a subclass of `int`, so `True + True == 2` — occasionally a subtle bug.

Division — which operator yields which type

```
6 / 2                # 3.0   "/" is TRUE division -> ALWAYS float
7 // 2               # 3     "/" floor division -> int if both int
7.0 // 2             # 3.0   ...float if either operand is float
-7 // 2              # -4    floors toward -infinity
int(-7 / 2)          # -3    truncates toward zero <- differs from // !
-7 % 3               # 2     modulo sign follows the DIVISOR in Python
divmod(17, 5)        # (3, 2) = (17 // 5, 17 % 5)
2 ** 10              # 1024  int; 2 ** 0.5 -> float
pow(2, 10, 1000)     # 24    fast modular exponentiation
round(2.5)           # 2     banker's rounding (half to EVEN): round(3.5) -> 4
```

`divmod` is the clean flat-index <-> grid-coordinate converter:

```
r, c = divmod(idx, cols)    # flat -> (row, col)
idx = r * cols + c          # (row, col) -> flat
```

Collection conversions — one rule

Every collection constructor takes any iterable, so a cast is just `Target(iterable)`.

From → To	How	Why you'd do it
list → set	<code>set(xs)</code>	dedup + O(1) membership (loses order & dups)
list → tuple	<code>tuple(xs)</code>	make it hashable — dict key, set element, memo key
set/dict → list	<code>list(s)</code> , <code>sorted(s)</code>	need indexing, ordering, or sorting
str → list	<code>list(s)</code>	mutate characters (strings are immutable)
chars → str	<code>"".join(chars)</code>	rebuild the string
ints → str	<code>"".join(map(str, nums))</code>	join requires strings
digits str → ints	<code>[int(c) for c in s]</code>	per-digit work
dict → pairs	<code>list(d.items())</code>	iterate/sort key-value pairs
pairs → dict	<code>dict(pairs)</code>	build a lookup
two lists → dict	<code>dict(zip(keys, vals))</code>	map one list onto another

```
list("abc")           # ['a', 'b', 'c']
set([1, 1, 2])        # {1, 2}
list(d)               # dict -> list of KEYS (not items!)
a, b = zip(*pairs)    # "unzip" -> two tuples
list(zip(*matrix))    # transpose rows <-> cols
```

Element-wise vs container conversion differ: `list(map(int, tokens))` converts each element; `tuple(xs)` converts the container.

2. String Methods

```
s.strip()              # trim whitespace ends (or chars: s.strip(".", ""))
s.lstrip(); s.rstrip()
s.split()              # split on any whitespace, drop empties
s.split(",")           # split on a delimiter (keeps empties)
s.split(",", 1)        # limit splits; s.rsplit(",", 1) from the right
s.splitlines()         # split on line boundaries
", ".join(parts)       # parts must all be strings
s.replace("a", "b")    # replace all (add count arg to limit)
s.find("x")            # index or -1 (safe)
s.index("x")           # index or ValueError
s.count("ab")          # non-overlapping occurrences
s.startswith("pre")    # also accepts a tuple of options
s.endswith(("py", ".txt"))
s.lower(); s.upper(); s.title(); s.capitalize(); s.swapcase()
s.isdigit(); s.isalpha(); s.isalnum(); s.isspace(); s.islower()
s.zfill(5)             # "42" -> "00042"
s.ljust(8); s.rjust(8); s.center(8) # pad to a width
```

Strings are immutable, so every method returns a new string. To edit characters, go `list(s)` -> mutate -> `"".join(...)`.

3. Slicing

```
a[start:stop:step]    # stop is EXCLUSIVE
a[2:5]               # indices 2, 3, 4
a[:3]; a[3:]         # head / tail
a[-1]; a[-2]         # from the end
a[::-1]              # reversed copy
a[::2]               # every other
a[:]                 # shallow copy of the whole sequence
s[i:j]               # substrings slice the same way
a[1:3] = [9, 9, 9]    # slice assignment (lists) – can change length
del a[1:3]            # delete a slice
```

Out-of-range slice bounds are clamped (no error): `a[:100]` is safe, while `a[100]` raises `IndexError`.

4. Sorting in Depth

```
sorted(xs)                                # new list; xs.sort() sorts in place
sorted(xs, reverse=True)
sorted(words, key=len)                    # by a derived key
sorted(pts, key=lambda p: (p[0], -p[1]))  # multi-key, mixed direction
# sort is STABLE: equal keys keep input order (sort by secondary, then primary)

from operator import itemgetter, attrgetter
sorted(rows, key=itemgetter(1, 0))         # by column 1, then column 0
sorted(objs, key=attrgetter("age"))

from functools import cmp_to_key
# "Largest Number": order so concatenation is biggest
nums = sorted(map(str, [3, 30, 34, 5, 9]),
               key=cmp_to_key(lambda a, b: (a + b < b + a) - (a + b > b + a)))
# -> ['9', '5', '34', '3', '30']
```

5. Math & Numerics

```
import math
math.gcd(12, 18)    # 6      (math.lcm in 3.9+)
math.lcm(4, 6)      # 12
math.isqrt(17)      # 4      integer sqrt, no float error
math.floor(2.7); math.ceil(2.1)
math.factorial(5)   # 120
math.comb(5, 2)     # 10     nCr (math.perm for nPr)
math.inf            # same as float("inf")
math.isnan(x); math.isclose(a, b)         # safe float comparisons

abs(-5); pow(2, 10); pow(2, 10, 1000)     # modular pow
(5).bit_length()    # 3
(7).bit_count()     # 3      set-bit count (3.10+); else bin(7).count("1")
```

Two Python-specific points to say out loud: **ints are arbitrary precision** — they never overflow (unlike Java/C++), and floats are IEEE-754 doubles, so `0.1 + 0.2 != 0.3` — compare with `math.isclose` or use integer/ `Decimal` math when exactness matters.

6. Unpacking & Argument Forms

```
a, b = b, a          # swap, no temp
first, *rest = seq    # rest is a list
*init, last = seq
a, (b, c) = 1, (2, 3) # nested

merged = [*a, *b]      # list splat
combined = {**d1, **d2} # dict merge (later keys win)
f(*args, **kwargs)      # spread into a call

def f(a, b, /, c, *, d): # "/" = positional-only, "*" = keyword-only
    ...
# a, b must be positional; d must be passed by name
```

7. Copying, Scope & Hashability

```
import copy
shallow = copy.copy(obj)      # or list[:], dict.copy(), set.copy()
deep    = copy.deepcopy(obj)  # independent nested structures

grid = [[0] * 3] * 3          # WRONG: 3 references to one row
grid = [[0] * 3 for _ in range(3)] # right
```

Scope (LEGB) — names resolve Local -> Enclosing -> Global -> Built-in. `global x` and `nonlocal x` let you rebind names in outer scopes.

Hashability — usable as dict keys / set members: immutables (`int`, `float`, `str`, `bool`, `frozenset`, and tuples of hashables). NOT hashable: `list`, `dict`, `set`. That's why you cast a list to a tuple to use it as a memo key or a visited-state marker.

8. OOP — Deeper

property with setter and deleter

```
class Temp:
    def __init__(self, c=0):
        self._c = c
    @property
    def c(self):          # getter: t.c
        return self._c
    @c.setter
    def c(self, value):   # setter: t.c = 5
        if value < -273.15:
            raise ValueError("below absolute zero")
        self._c = value
    @c.deleter
    def c(self):          # del t.c
        del self._c
```


str vs repr

```
class P:
    def __init__(self, x):
        self.x = x
    def __repr__(self):          # unambiguous, for devs/debugging/REPL
        return f"P(x={self.x})"
    def __str__(self):          # readable, for print() / users
        return f"point {self.x}"
# print(p) uses __str__; the REPL and repr(p) use __repr__. Define __repr__ at minimum.
```

Dunders that make objects behave like built-ins

```
class Bag:
    def __init__(self):
        self._d = {}
    def __len__(self):          return len(self._d)          # len(bag)
    def __getitem__(self, k):   return self._d[k]           # bag[k]
    def __setitem__(self, k, v): self._d[k] = v             # bag[k] = v
    def __contains__(self, k):  return k in self._d         # k in bag
    def __iter__(self):         return iter(self._d)        # for x in bag
    def __call__(self, k):      return self._d.get(k)       # bag(k) - callable
```

Get all comparisons from two methods

```
from functools import total_ordering

@total_ordering
class V:
    def __init__(self, n):
        self.n = n
    def __eq__(self, o): return self.n == o.n
    def __lt__(self, o): return self.n < o.n
    # total_ordering supplies <=, >, >=, !=
```

The eq / hash contract

```
# Defining __eq__ makes the class UNHASHABLE unless you also define __hash__.
class Pt:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __eq__(self, o):
        return (self.x, self.y) == (o.x, o.y)
    def __hash__(self):          # required to put Pt in a set/dict
        return hash((self.x, self.y))
```

new vs init (allocation vs initialization)

```
class Singleton:
    _inst = None
    def __new__(cls):
        if cls._inst is None:
            cls._inst = super().__new__(cls) # allocation happens here
        return cls._inst
    def __init__(self):
        ... # runs on every call – init carefully
```

Encapsulation conventions

```
self.public # part of the API
self._internal # convention: "don't touch" (NOT enforced)
self.__mangled # name-mangled to _Class__mangled (avoids subclass clashes)
# Python has no truly private members.
```

MRO & multiple inheritance

```
class A: ...
class B(A): ...
class C(A): ...
class D(B, C): ... # diamond
D.__mro__ # D -> B -> C -> A -> object (C3 linearization)
# super() follows the MRO, not just the literal parent -> cooperative inheritance
```

Duck typing and Protocol (structural interfaces)

```
# "If it has the methods, it works" – no explicit interface/inheritance needed.
from typing import Protocol

class Comparable(Protocol):
    def __lt__(self, other) -> bool: ...
# Any class with __lt__ satisfies Comparable – structural, not nominal typing.
```

Composition over inheritance

```
# Prefer has-a (composition) over is-a (inheritance) when sharing behavior, not identity.
class Engine: ...
class Car:
    def __init__(self):
        self.engine = Engine() # swappable, looser coupling
```

Runtime checks, the class-mutable trap, and richer dataclasses

```
isinstance(x, (int, float))      # tuple = "any of"
issubclass(Savings, Account)

class Bad:
    items = []                    # SHARED across all instances – mutating leaks!
class Good:
    def __init__(self):
        self.items = []         # per-instance

from dataclasses import dataclass

@dataclass(frozen=True)          # immutable + hashable (usable as a dict key)
class Point:
    x: int
    y: int
    def __post_init__(self):      # validation after the auto __init__
        if self.x < 0:
            raise ValueError("x must be non-negative")

from typing import NamedTuple

class Color(NamedTuple):         # typed, immutable, lightweight record
    r: int
    g: int
    b: int
```

9. File Access, Processing & Error Handling

```
# Reading – `with` guarantees the file closes, even on exception
with open("data.txt", encoding="utf-8") as f:
    text = f.read()              # whole file as one string
    # f.readlines()              # list of lines (each keeps its "\n")
    # for line in f: ...          # lazy, line-by-line – best for big files

# Writing / appending
with open("out.txt", "w", encoding="utf-8") as f:  # "w" truncates
    f.write("hello\n")
    f.writelines(lines)          # does NOT add newlines for you

# Strip the trailing newline while processing
with open("data.txt") as f:
    rows = [line.rstrip("\n") for line in f]
```

File modes:

"r"	read (default)	"w"	write (truncate)	"a"	append
"x"	create, fail if exists			"+"	read + write
"b"	binary (rb / wb)	"t"	text (default)		

`pathlib` for paths and one-shot read/write:

```

from pathlib import Path
p = Path("data") / "input.txt"    # OS-safe join
p.exists(); p.is_file(); p.is_dir()
text = p.read_text(encoding="utf-8") # read in one call
p.write_text("...")                # write in one call
p.name; p.stem; p.suffix; p.parent # "input.txt", "input", ".txt", Path("data")
list(Path(".").glob("*.py"))       # pattern search

```

JSON and CSV:

```

import json
with open("config.json") as f:
    cfg = json.load(f)          # file -> object
with open("out.json", "w") as f:
    json.dump(cfg, f, indent=2) # object -> file
obj = json.loads(s)            # string <-> object (json.dumps to serialize)

import csv
with open("data.csv", newline="") as f: # newline="" is required for csv
    for row in csv.DictReader(f):       # each row is a dict keyed by header
        ...
with open("out.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["name", "score"])
    w.writerows(rows)

```

File error handling — catch the specific failure:

```

from pathlib import Path

try:
    text = Path("maybe.txt").read_text(encoding="utf-8")
except FileNotFoundError:
    text = "" # sensible default
except PermissionError:
    raise # can't fix it here
except UnicodeDecodeError:
    text = Path("maybe.txt").read_text(errors="replace")
except OSError as e: # base class for I/O failures
    raise

```

Exception	Raised when
<code>FileNotFoundError</code>	path doesn't exist (read mode)
<code>PermissionError</code>	no read/write permission
<code>IsADirectoryError</code>	opened a directory as a file
<code>FileExistsError</code>	"x" mode on an existing file
<code>UnicodeDecodeError</code>	bytes don't decode with the given encoding
<code>OSError</code>	base class for all of the above

The `with` block is the idiomatic guarantee the handle closes; the manual equivalent is `try/finally` with `f.close()`.

10. I/O & Input Parsing (online-assessment settings)

```
n = int(input())                                # one int on a line
a, b = map(int, input().split())                # two ints on a line
nums = list(map(int, input().split()))          # a row of ints
grid = [input().split() for _ in range(rows)]

import sys
data = sys.stdin.read().split()                # fast bulk read; index through tokens
it = iter(data)
n = int(next(it))

print(*nums, sep=" ")                          # space-separated row
sys.stdout.write("\n".join(map(str, results))) # fast bulk output
```

CodeSignal / HackerRank / Codility OAs hand you raw stdin and make you parse it — directly relevant to OA-format rounds. For large inputs, `sys.stdin` beats repeated `input()`.

11. Smaller Things Worth Knowing

f-string format mini-language:

```
f"{x:.2f}"      # 2 decimals
f"{n:04d}"      # zero-pad to width 4
f"{n:,}"        # thousands separators
f"{n:b}"        # binary (also :x hex, :o octal)
f"{s:<8}"        # left-pad (also :>8 right, :^8 center)
f"{r:.1%}"      # percentage
```

```
from operator import itemgetter, attrgetter, add
from functools import partial
inc = partial(add, 1)                # inc(5) -> 6

[1, 2] < [1, 3]                      # True – lists/tuples compare lexicographically

from collections import ChainMap
cfg = ChainMap(overrides, defaults)  # layered lookup, first match wins

from contextlib import suppress
with suppress(KeyError):             # ignore a specific exception cleanly
    del d[k]

import re
re.findall(r"\d+", s)                # all digit-runs
re.sub(r"\s+", " ", s)               # collapse whitespace
re.match(r"^[a-z]+$", s)             # anchored match -> Match or None
```